Carleton University
School of Computer Science

Last updated: February 6, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS TUTORIAL 5

Tasks/Questions (Part A)

1. Compile and run 3000test.c. It takes a filename as an argument and reports information on the file. Try giving it the following and see what it reports:
   - a regular text file that exists
   - a directory
   - a symbolic link
   - a device file (character or block)

   It reports the number of 'a' if the file is a regular file or the link points to a regular file. The length is the logical size.

2. Change 3000test to use `lstat()` rather than stat. How does its behavior change?
   `lstat()` does not resolve (dereference) a symbolic link. So, it returns the inode of the link itself.

3. Modify 3000test so when it is given a symbolic link it reports the name of the target. Use `readlink(2)` (this notation describes how to access the man page, e.g., `man 2 readlink`).
   First, use `S_ISLNK` to check if it's a symbolic link and then `readlink()`.

4. Are there files or directories that you cannot run 3000test on? Can you configure file/directory permissions so as to make something inaccessible to 3000test? Note that this is twofold: 1) whether it is completely inaccessible to 3000test (**nothing can be displayed**, except the error msg); 2) whether it can be accessed for the search (**no access to file content**).
   This is where you can improve your understanding of the relationship between inodes and directory entries, and between the inodes and their data blocks.
   The permission bits are in the inode, so denied access only applies to data blocks but not the inode.
   Likewise, if the directory entry is no accessible (which is like data of the directory inode), there's no way to find the inode number, not to mention what permission bits are there.

Tasks/Questions (Part B)

1. Run `dd if=/dev/zero of=foo bs=8192 count=32K` and use the `ls` command to check it. What is the logical size of the file foo? How much space does it consume on disk? (Hint: Look at the size option to ls). In comparison, create another file with `dd if=/dev/zero of=foo2 bs=8192 seek=31K count=1K`. Any observations?
   This is the case when logical size can be greater than physical size ("holes" in a file). The first command explicitly writes that many zero's taking up physical space. The second command skips 31K blocks writing only 1K blocks (note: the skipped blocks are still part of

the file size, e.g., if you write to the second byte, there must exist the first byte), where the file system does some optimization by only allocating 1K blocks, hence the holes.

2. Run `mkfs.ext4 foo`. Does foo consume any more space or less? Do the same (`mkfs.ext4 foo2`) to the other file and answer the same question.
   `mkfs` creates the file system structures (e.g., the superblocks) so it writes data to the file, leading to actual allocation of space. This may trigger the optimization (so the OS realizes that there are repeated 0's), which may potentially make the file smaller (for foo). But for foo2, since it's already optimized (with holes, no wasted space), extra physical space needs to be allocated to hold what `mkfs` writes.

NOTE: on certain versions of Ubuntu, you may need to use:

3. Create any file in `/mnt` (e.g., `sudo touch test.txt`) and run `sudo mount foo /mnt`. Do you still see the file you just created?
   sudo mount -o loop foo /mnt
   As discussed, mounting over an existing directory (it has to be existing) will not affect its current content. It's just like redirection or pointing to another file system from there.

the -o loop explicitly tells mount to use a loopback dev

4. Run `df`. What device is mounted on `/mnt`? What is this device?
   You should see the loopback device mounted on `/mnt`.
   Otherwise mount will fail

5. Run `sudo umount /mnt`. What have gone away and what is back?
   See 3 above. The previously existing but hidden files will be back.

6. Run `dd if=/dev/zero of=foo conv=notrunc count=10 bs=512`. How does the "`conv=notrunc`" change dd's behavior (versus the command in question 2)?
   `notrunc` means, instead of erasing an existing file first (truncating to zero), `dd` will just overwrite wherever it writes, leaving anywhere not written untouched. This way, you still have the file system created previously but have overwritten part of the primary superblock.

7. Run `sudo mount foo /mnt`. What error do you get?
   Well, the file system has been corrupted in the previous step (superblock).

8. What command can you run to make foo mountable again? What characteristic of the file system enables this command to work? (Hint: revisit the instructions of this tutorial if you have no idea)
   `fsck` finds the backup superblocks and replaces the corrupted primary one with it.

`